Tecnológico Superior de Jalisco, Arandas.

Carrera: ISIC

Proyecto final

Ingeniería de software

Documentación

Presenta:

Daniela Sepúlveda Gómez núm. :220110071.

Trabajador académico:

Rosa María Chávez Camarena.

Índice

1. Introducción 3

2. Objetivo del proyecto 4

3. Antecedentes del problema 4

4. Descripción de la situación actual 5

5. Normas y referencias 6

5.1 Disposiciones legales y normas aplicadas (licencias) 7

5.3 Métodos, Herramientas, Modelos, Métricas(certificaciones) y Prototipos . 8

5.3.1. Métodos y Herramientas 8

5.3.2. Modelos, Métricas y Prototipos 10

6. Definiciones y abreviaturas 17

7. Requisitos iniciales 18

Diseño de la base de datos 20

Proyecto asesorado por ingenieros e ingenieras con experiencia 21

8. Alcance 21

9. Limitaciones o restricciones (análisis del modelo) 22

10. Estudio de alternativas y viabilidad (estudio de factibilidad) 23

Justificación 23

-Expertos 24

-Usuarios 29

Reporte de conclusión 32

11. Descripción de la solución propuesta (Cronograma de actividades, paso a paso explicando el front-end y el Back-end) 33

Base de datos 34

12. Análisis de Riesgos 51

13. Bibliografías 53

1. Introducción

El presente documento describe el desarrollo del sistema llamado “Abarrotes Silvia”, una solución tecnológica para los problemas identificados en el negocio, creado para optimizar la gestión de productos, ventas y compras.

Este proyecto lo desarrollé como parte de una práctica profesional del semestre, de hecho, es el proyecto integrador, el más importante. Cuenta y se califica con tres materias del programa, prácticamente las más importantes: programación web, ingeniería de software y administración de base de datos.

La clienta quien es la gerente general, requería una plataforma digital, específicamente en la nube, que permitiera llevar un control más eficiente de sus operaciones diarias.

Con este fin, diseñé una aplicación web funcional, intuitiva, interfaz con diseño amigable y moderno, mejorando la experiencia del usuario y además visualmente atractiva. Cubre los principales procesos administrativos del negocio: el manejo de usuarios, clientes, proveedores, productos, ventas, compras, caja, reportes y más.

El sistema está pensado en facilitar las tareas del personal de la tienda, especialmente en el modo administrador, el cual permite registrar todo de manera rápida, consultar reportes en tiempo real y administrar el inventario de forma precisa; la clienta pidió eso en especial, llevar un control sobre el inventario, ya que, una de las personas más importantes en el negocio es una adulta mayor, provocando así sus complicaciones con el manejo del negocio.

Es por eso que este proyecto lo realicé en base a los requerimientos pedidos por la clienta.

2. Objetivo del proyecto

Desarrollar una aplicación web intuitiva, amigable y funcional para la tienda de “Abarrotes Silvia”, que permita administrar eficientemente los procesos y registros de ventas, compras, control de productos, proveedores y generación de reportes, con el fin de mejorar la organización, reducir errores manuales y optimizar la toma de decisiones en la gestión del negocio.

3. Antecedentes del problema

La tienda de “Abarrotes Silvia” realizaba sus operaciones de forma manual, utilizando libretas y calculadoras, incluso en ocasiones mentalmente sin anotar nada, para llevar el registro de ventas, compras e inventario. Esta forma de trabajo, aunque funcional en sus inicios, se volvió ineficiente con el crecimiento del negocio y al aumento en la variedad de productos y volumen de transacciones.

El control de inventario se manejaba de manera muy simple, lo cual dificultaba conocer con precisión los niveles de productos, generando como problemas faltantes de productos y exceso de mercancía. Asimismo, no existía una forma clara de generar reportes de finanzas, como por ejemplo saber las ganancias al día o saber qué porcentaje y dinero se está ganando en cada uno de los productos, de cada venta, lo que afectaba la toma de decisiones y planificación del negocio.

La falta de un sistema también limitaba la velocidad en dichos procesos ya mencionados, aumentando el riesgo de errores y pérdida de información importante.

Esta situación evidenció la necesidad de implementar una solución tecnológica que permitiera automatizar y optimizar el negocio.

4. Descripción de la situación actual

Actualmente la tienda “Abarrotes Silvia” cuenta con un sistema web llamado VentaSmart, desarrollado específicamente para cubrir las necesidades de gestión del negocio. Esta aplicación permite a los usuarios registrar ventas, controlar compras a proveedores, manejar el inventario de productos y generar reportes detallados en tiempo real.

El sistema ha reemplazado los registros manuales, mejorando significativamente la organización y reduciendo los errores humanos. Cada módulo está enfocado en una función específica: ventas, caja, clientes, proveedores, productos, reportes y control de usuarios. La interfaz está diseñada de forma intuitiva, moderna y atractiva, facilitando su uso por parte del personal, incluso sin conocimientos técnicos avanzados.

Además, se ha logrado tener un control más exacto del inventario, automatizar los cálculos de precios, ganancias y obtener estadísticas visuales que apoyan a la toma de decisiones. La digitalización de los procesos ha incrementado la productividad del negocio y ha permitido ofrecer una atención más rápida y eficiente tanto a los clientes como a los usuarios.

5. Normas y referencias

Normas aplicadas:

Normas de desarrollo web:

- Uso de buenas prácticas en HTML5, CSS3 y JavaScript para el diseño de interfaces web.
- Estructuración del proyecto conforme a los principios de diseño responsive y accesibilidad.

Normas de documentación:

- Organización de la documentación basada en el formato técnico otorgado por el equipo de profesores de sistemas (Rosa María Chávez Camarena, Rocio Parra y Francisco González).
- Redacción clara y concisa siguiendo criterios de estilo académico.

Normas de bases de datos:

- Modelado de base de datos siguiendo las reglas de normalización hasta la 3FN (Tercera Reforma Normal).
- Uso de claves primarias y foráneas para garantizar integridad referencial.

Referencias:

MDN Web Docs. (s.f.). HTML, CSS y JavaScript documentation. Recuperado de: https://developer.mozilla.org/

W3Schools. (s.f.). Tutoriales de desarrollo web. Recuperado de: https://www.w3schools.com/

MySQL Documentation. (s.f.). MySQL Reference Manual. Recuperado de: https://dev.mysql.com/doc/

Bootstrap. (s.f.). The most popular HTML, CSS, and JS library. Recuperado de: https://getbootstrap.com/

ISO/IEC 25010:2011. Modelo de calidad del software (referido para criterios de usabilidad y funcionalidad).

5.1 Disposiciones legales y normas aplicadas (licencias)

- PHP – Licencia PHP License (OSI Approved), permite su uso y distribución gratuita.
- MySQL – Licencia GPL (General Public License), libre para proyectos propios o académicos.
- Bootstrap – Licencia MIT, permite su uso comercial y académico sin restricciones.
- jQuery y Chart.js – Ambas bajo licencia MIT, lo que habilita su uso sin costo.
- SweetAlert2 – También bajo licencia MIT, utilizada para ventanas de alerta interactivas.
- Font Awesome (si aplica) – Bajo licencia CC BY 4.0, se permite su uso con atribución.

Publicación en línea:

- Condiciones de uso y políticas del proveedor de hosting, llamado sistemos.site, especialmente en cuanto al contenido alojado y la privacidad de dato.
- Configuración segura del entorno de producción, con el fin de proteger credenciales y datos sensibles que el sistema pueda manejar.

Protección de datos:

- Proteger el acceso al sistema mediante autenticación por usuario y contraseña.
- Restringir funciones administrativas solo a usuarios autorizados.
- Mantener segura la información de ventas, productos y usuarios registrados.

5.3 Métodos, Herramientas, Modelos, Métricas(certificaciones) y Prototipos

5.3.1. Métodos y Herramientas

Durante del desarrollo del sistema, se aplicaron diversos métodos de trabajo, se utilizaron herramientas y modelos apropiados para garantizar un desarrollo eficiente, organizado y funcional:

Herramientas:

Editor de código: Visual Studio Code, por su ligereza y amplia compatibilidad con lenguajes web.

Frameworks y bibliotecas: Bootstrap 5 para la interfaz responsiva, JQuery para manipulación DOM, Chart.js para gráficos interactivos, SweetAlert2 para alertas modernas.

Base de datos: MySQL, gestionada desde la terminal y phpMyAdmin.

Servidor web: Hostinger, contratado por varios del grupo.

Métodos:

Desarrollo incremental: El sistema fue construido por partes funcionales (productos, ventas, compras, reportes), permitiendo pruebas frecuentes y mejoras continuas.

Metodología ágil (Scrum): Se aplicó de forma básica, definiendo tareas semanales como “sprints”, con seguimiento del avance y retroalimentación constante por parte del usuario final (mi clienta, gerente de la tienda).

Scrum es una metodología ágil que se utiliza en la gestión de proyectos, especialmente en el desarrollo de software. Es una palabra proveniente del rugby que

significa "formación de equipo compacta", lo que representa el enfoque en el trabajo en equipo en el proceso de desarrollo de proyectos.

Scrum ofrece un marco que permite a los equipos de desarrollo mantenerse enfocados en los requisitos del cliente, a través de una gestión iterativa y flexible. Se basa en tres pilares fundamentales: la transparencia, la inspección y la adaptación. Scrum es un marco ágil de trabajo que permite gestionar proyectos de manera flexible y eficiente. Se basa en ciclos iterativos llamados sprints, donde los equipos trabajan en entregas incrementales de producto. Es ideal para proyectos que requieren adaptabilidad, como el desarrollo de software.

Principales Elementos de Scrum

1. Roles en Scrum

Scrum Master: Facilita el proceso, elimina obstáculos y asegura que el equipo siga las prácticas de Scrum.

Product Owner: Representa a los clientes/usuarios y gestiona el Product Backlog (lista priorizada de requisitos).

Development Team: Grupo de profesionales que trabajan en la implementación de los requisitos del producto.

2. Artefactos de Scrum

Product Backlog: Lista priorizada de tareas o funcionalidades a desarrollar.

Sprint Backlog: Conjunto de tareas seleccionadas del Product Backlog que se

completarán en el sprint actual.

Incremento: Producto funcional al final de cada sprint.

3. Eventos en Scrum

Sprint: Iteración de trabajo de 1 a 4 semanas en la que se desarrolla un incremento del producto.

Sprint Planning: Reunión para planificar el trabajo del sprint.

Daily Scrum: Reunión diaria (máx. 15 min) para sincronizar avances y detectar bloqueos.

Sprint Review: Presentación del incremento al Product Owner y otros interesados.

Sprint Retrospective: Evaluación del sprint para mejorar procesos en el siguiente ciclo.

- Beneficios de Scrum
- Flexibilidad y adaptación a cambios.
- Entregas frecuentes y funcionales.
- Mayor colaboración entre el equipo y el cliente.
- Transparencia y mejora continua.

Enfoque iterativo: Se realizaron versiones preliminares (prototipos) de cada módulo, que luego fueron evaluadas y mejoras.

5.3.2. Modelos, Métricas y Prototipos

Métricas:

Aunque no se utilizaron métricas certificadas, se aplicaron criterios básicos para medir el funcionamiento y calidad del sistema.

Correcta funcionalidad de los módulos: Cada módulo fue probado individualmente y en conjunto para asegurar su integridad.

Rendimiento del sistema en entorno real (carga rápida, navegación fluida).

Interfaz amigable y responsiva.

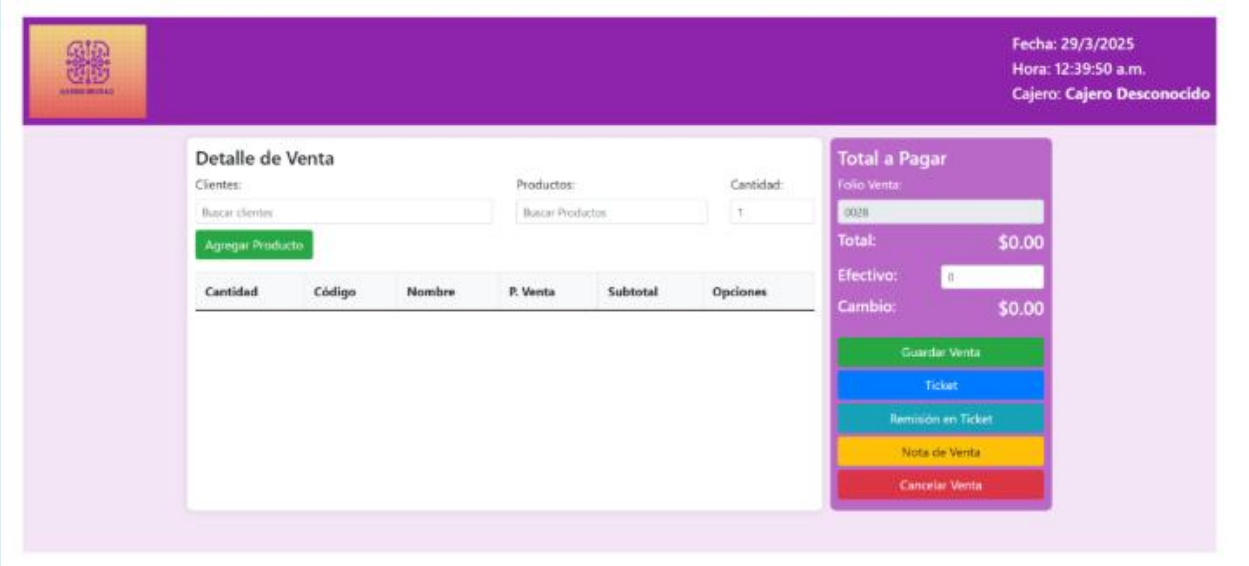
Usabilidad y experiencia del usuario final.

Prototipos:

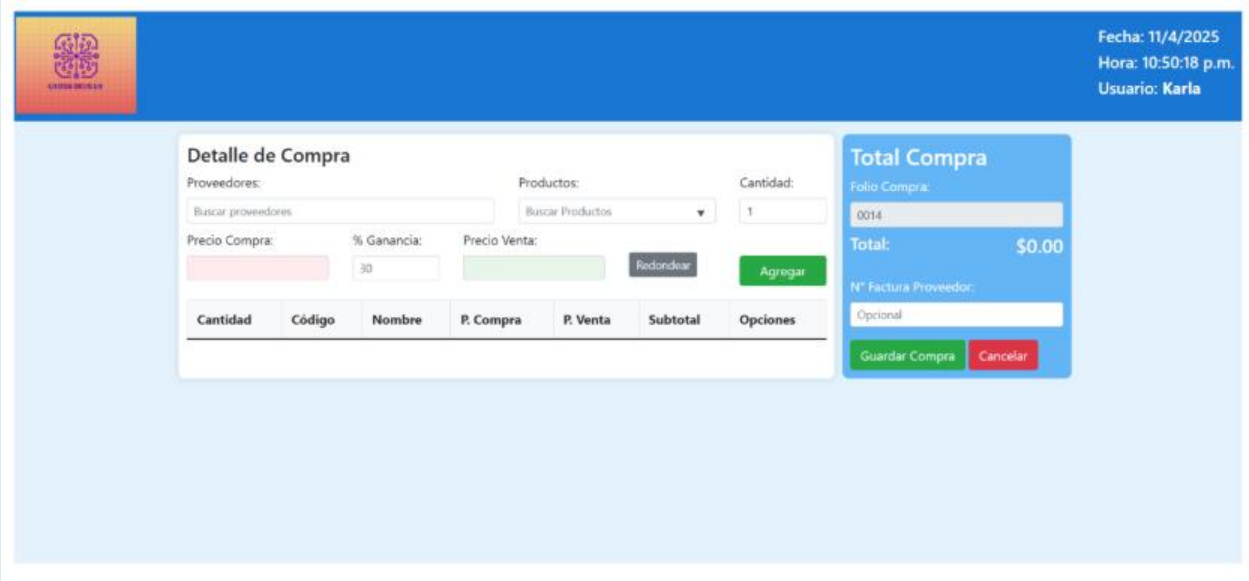
Antes del desarrollo definitivo, se elaboraron bocetos y maquetas visuales de las principales interfaces del sistema, como:

Pantalla login (inicio de sesión).

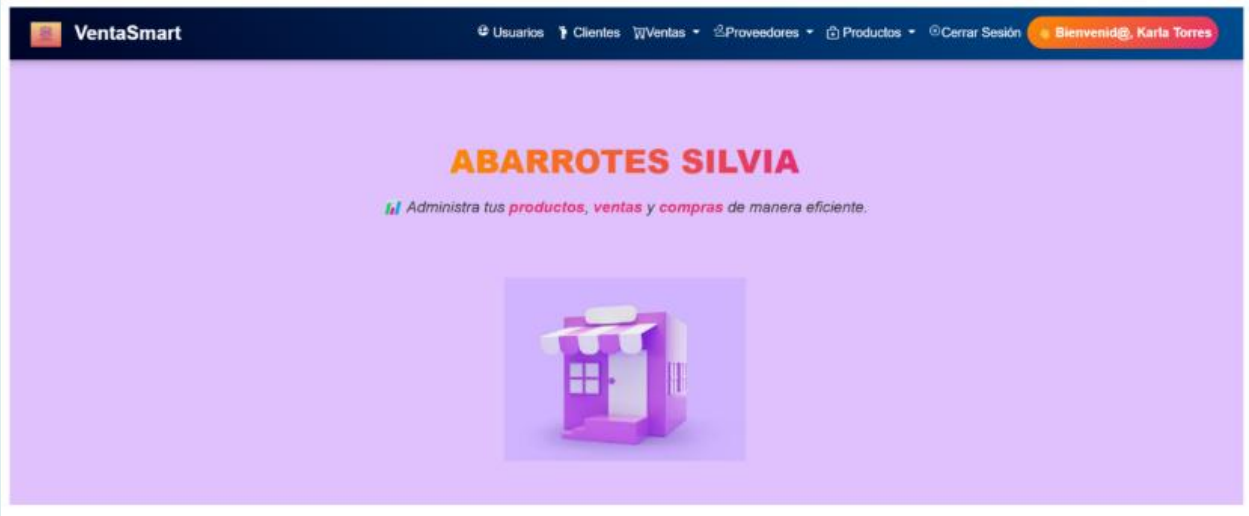
Panel de ventas.



Compras.

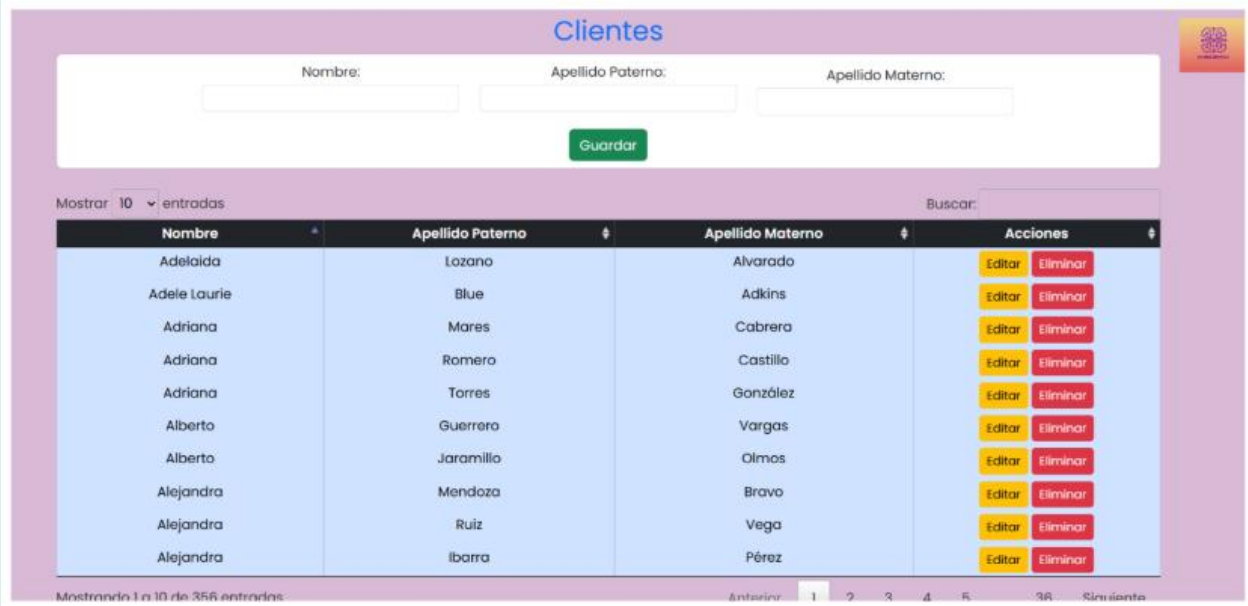


Menú.



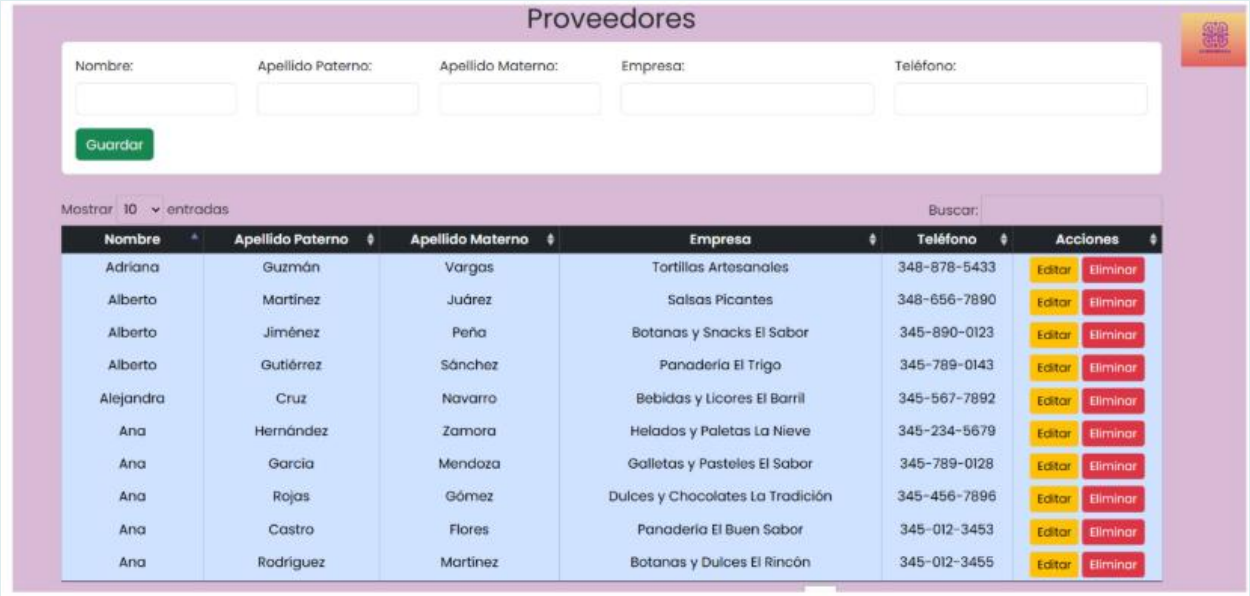
Gestión de usuarios.

Clientes.

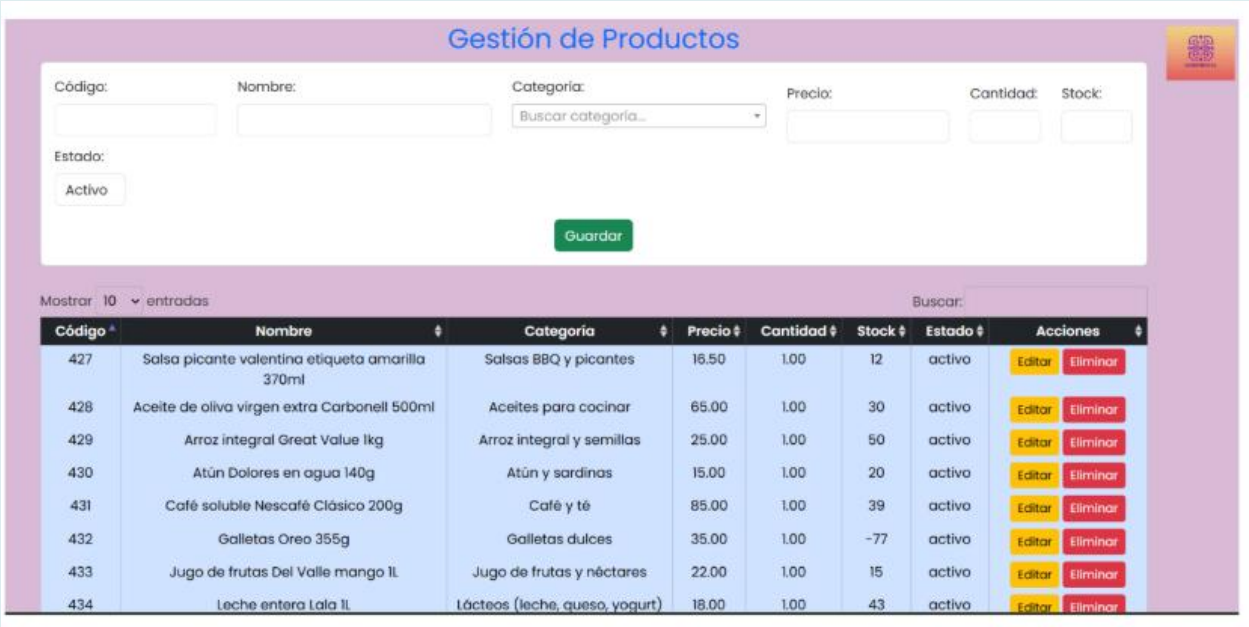


Consulta de ventas:

Proveedores.



Productos.



Categorías.

Modelos:

Modelo MVC básico (Modelo-Vista-Controlador): aunque no de forma estricta, se buscó separar la lógica del sistema (PHP), la presentación (HTML/CSS) y el control de datos (MySQL).

Modelo de entidad-relación (E-R): Se diseñó un modelo de base de datos relacional que incluye tablas como productos, ventas, det_venta, proveedores, compras_prov, entre otras, siguiendo principios de normalización.

Modelo de desarrollo incremental: Se abordó el sistema por módulos funcionales (productos, ventas, compras, reportes), permitiendo implementar y probar de forma parcial y continua.

6. Definiciones y abreviaturas

Sistema web: Aplicación accesible desde un navegador, alojada en un servidor.
Base de datos (BD): Conjunto organizado de información estructurada, almacenada electrónicamente.
PHP: Lenguaje de programación del lado del servidor usado para desarrollar el backend del sistema.
HTML: Lenguaje de marcado utilizado para estructurar el contenido de páginas web.
CSS: Lenguaje usado para definir el estilo visual de las páginas web.
JavaScript (JS): Lenguaje de programación utilizado para la interactividad del frontend.
MySQL: Sistema de gestión de bases de datos relacional utilizado en el proyecto.
CRUD: Anónimo de crear, leer, actualizar y eliminar: operaciones básicas en bases de datos.
UI (User IInterface): Interfaz de usuario: conjunto de pantallas, páginas y elementos visuales con los que interactúa el usuario.

UX (User Experience): Experiencia del usuario: percepción y respuesta del usuario al interactuar con el sistema.
MVC: Modelo de arquitectura de software que separa la lógica del negocio, la interfaz de usuario y el control de entrada.
Hosting: Servicio que permite alojar un sitio web para que sea accesible en Internet.

7. Requisitos iniciales

Requisitos funcionales

RF01: Interfaz de usuario fácil de usar.

RF02: Agregar un módulo para registrar compras a proveedores.

RF03: Ponerlo en la nube.

RF04: Crear una base de datos con sus respectivas tablas.

RF05: Crear un apartado donde se puedan visualizar y buscar los productos.

RF06: Llevar control sobre productos con importe.

RF07: Crear apartado de ganancias por día.

RF08: Registro de entradas y salidas manual.

RF09: Registrar ventas por día.

RF10: Notificar cuando a un producto le queden pocas existencias.

RF11: Dispositivo de entrada preferente en computadora.

Requisitos no funcionales:

RNF01: Acceso a internet.

RNF02: Eficiencia.

RNF03: Seguridad de los usuarios.

RNF04: Crear usuario administrador.

RNF05: Los permisos de acceso al sistema podrán ser cambiadas por al administrador.

RNF06: El sistema debe proporcionar mensajes de error a los usuarios.

RNF07: El sistema debe asegurar que los datos estén protegidos del acceso no autorizado.

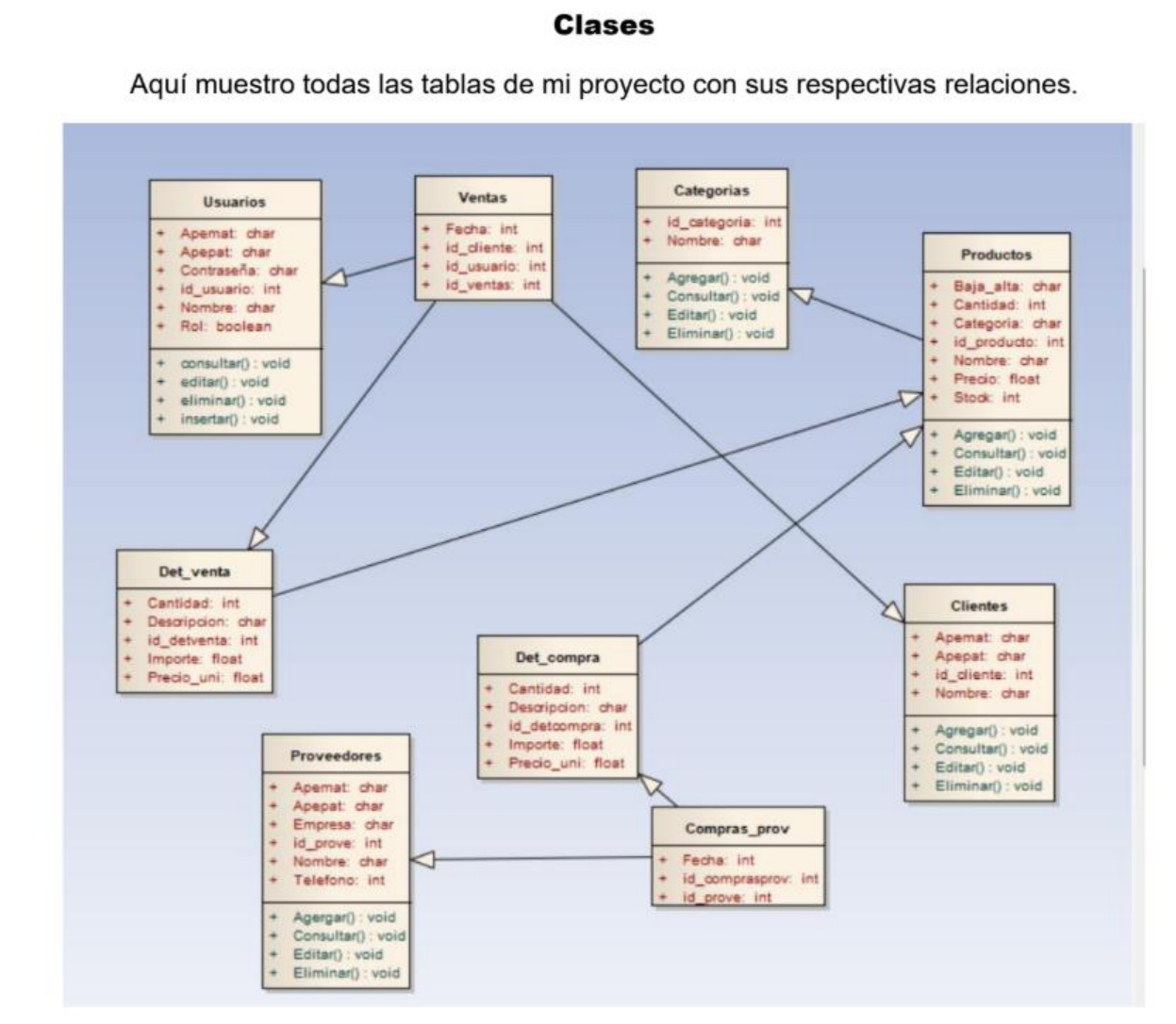
RNF08: Comprar un hosting.

RNF09: El sistema debe ser rápido.

RNF10: Portabilidad.

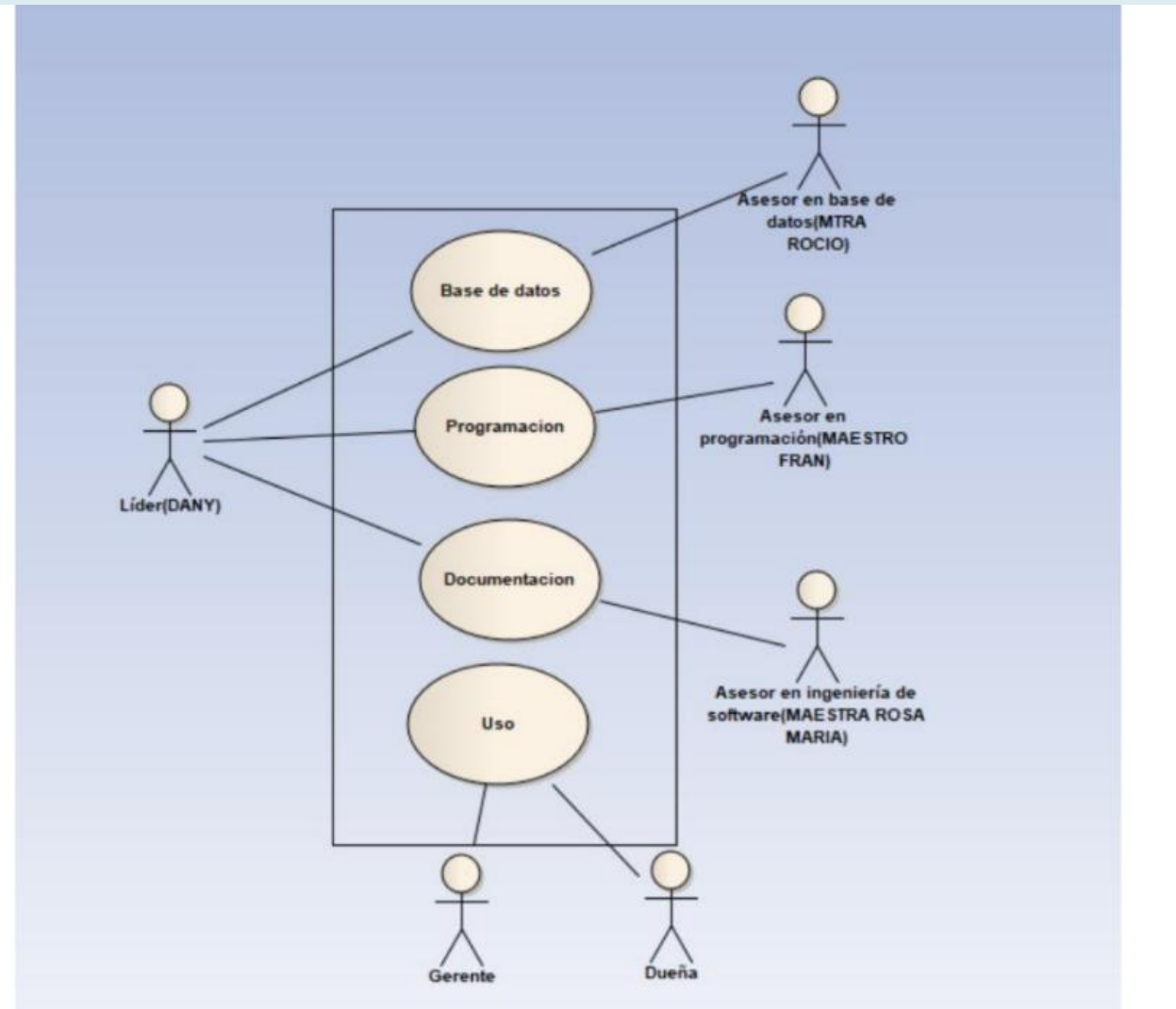
RNF11: Debe ser únicamente en idioma español latino.

Diseño de la base de datos



En los diagramas UML que realizamos en una clase, hice mi diseño de la base de datos relacional, son tablas vinculadas entre sí medianet FOREIGN KEYS y claves foráneas. Todas creadas para un sistema en específico.

Proyecto asesorado por ingenieros e ingenieras con experiencia



8. Alcance

El presente proyecto de software tiene como finalidad el desarrollo de un sistema web orientado a la gestión de una tienda de abarrotes llamada “Abarrotes Silvia”, que permita automatizar procesos clave como el control de inventario, el registro de ventas y compras, así como la generación de reportes de ganancias.

Está diseñado específicamente para cubrir las necesidades operativas del negocio, facilitando su administración diaria.

Funcionalidades de alcance:

- Registro y administración de productos, con precios de costo y venta.
- Control de inventario automático, actualizado con cada compra o venta.
- Registro de compras a proveedores y cálculo del precio venta con porcentaje de utilidad.
- Módulo de ventas con detalle de productos, total por ticket y control de efectivo.
- Generación de reportes visuales (ingresos, ganancias, etc).
- Panel administrativo protegido por autenticación de usuario.
- Interfaz amigable, accesible desde navegadores web comunes.
- Diseño responsivo que permite su uso desde computadoras y dispositivos móviles.
- Implementación en hosting web para su disponibilidad remota.

9. Limitaciones o restricciones (análisis del modelo)

Se espera que varias de las siguientes limitaciones puedan ser implementadas en un futuro, como alguna actualización del sistema.

- El sistema no incluye manejo de inventario por código de barras o escáner físico (aunque puede adaptarse en el futuro).
- El sistema no muestra un historial de ingresos y retiros, no permite consultarlos (aunque se puede implementar en el futuro).
- Está implementada como opción la búsqueda de consultas por cliente, pero no es funcional.
- No permite realizar diferentes tipos de pagos, solo efectivo.
- No cuenta con un módulo de créditos para los clientes.
- No se contempla integración con sistemas externos de facturación electrónica.
- Solo se implementan dos tipos de roles de usuario, empleado y administrador.

10. Estudio de alternativas y viabilidad (estudio de factibilidad)

Justificación

Este proyecto de software surge como una solución a la necesidad de la señora Silvia con un sistema eficaz y fácil de usar para la gestión de compras, ventas e inventario. actualmente, muchas tiendas de abarrotes enfrentan dificultades para llevar un control

detallado de sus productos y transacciones, lo que puede generar pérdidas, mermas y errores.

El desarrollo de esta plataforma permitirá a los vendedores mejorar su gestión de inventario, optimizar sus registros de ventas y compras, y tomar decisiones informadas sobre su negocio. Al contar con un sistema que notifica productos con pocas existencias y proporciona un registro detallado de ingresos y egresos, se mejora la eficiencia operativa y se minimizan los riesgos de mermas de productos o pérdidas financieras. Además, al ser un software basado en web, facilita el acceso remoto y la actualización en tiempo real de la información.

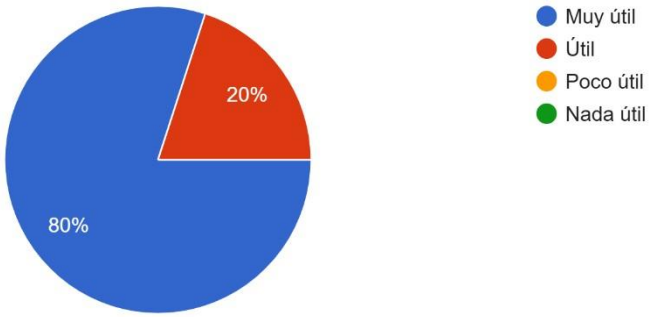
Estadísticas de las encuestas

-Expertos

Desarrolladores

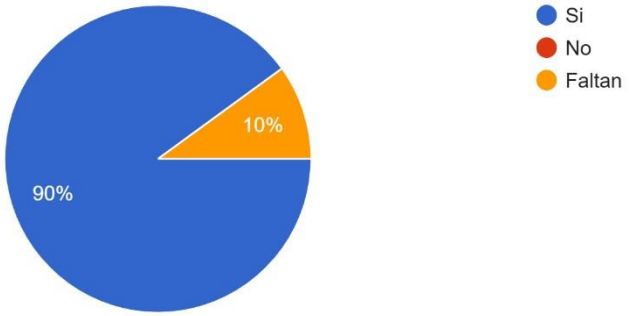
1-¿Cómo percibes la utilidad de este sistema para un negocio?

10 respuestas



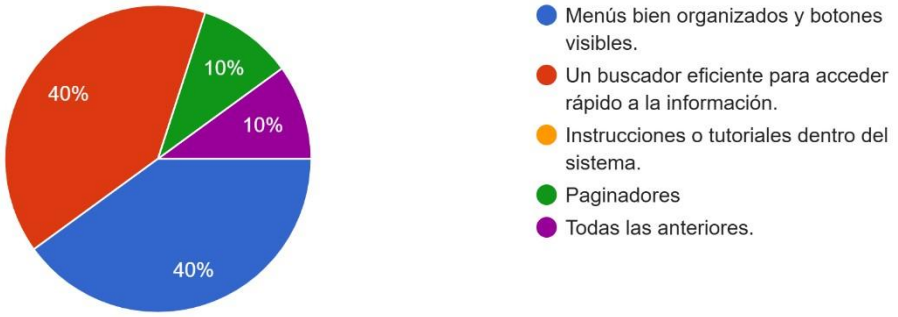
2-¿Crees que estos requisitos funcionales cubren las necesidades básicas de un negocio que maneja ventas y compras?

10 respuestas



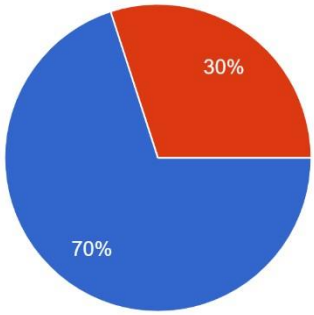
3- ¿Qué elementos consideras esenciales en la interfaz para facilitar la navegación?

10 respuestas



4-¿El apartado para visualizar y buscar productos debería incluir filtros avanzados?

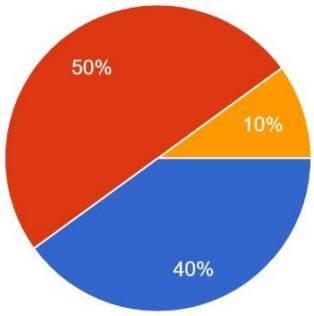
10 respuestas



- Sí, por categorías, precio, fecha de ingreso, etc.
- No, con un buscador básico es suficiente.
- No es necesario, ya que el inventario no es muy grande.

5- Para el control de productos con importe y ganancias diarias, ¿Qué funcionalidad te parecería más útil?

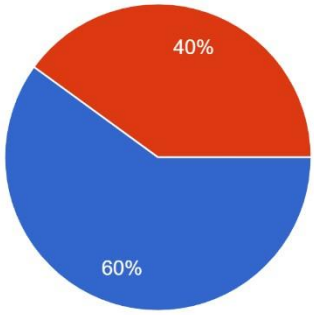
10 respuestas



- Un gráfico de ingresos y egresos.
- Un reporte detallado con exportación a Excel/PDF.
- Una alerta automática cuando se detecten anomalías en los ingresos.

6-¿Qué tan viable ves que este sistema esté instalado en un sitio web en lugar de una aplicación de escritorio?

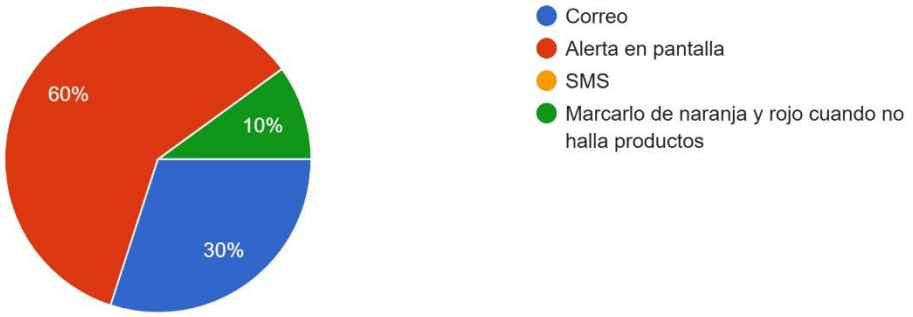
10 respuestas



- Muy viable, es más accesible desde cualquier lugar.
- Viable, pero debería tener opción de trabajar sin internet.
- Prefiero una aplicación de escritorio, es más segura y estable.

7- ¿Qué tipo de notificación crees que sería más efectiva cuando un producto tenga pocas existencias?

10 respuestas



8- ¿Crees que el sistema debería permitir diferentes niveles de acceso para usuarios (por ejemplo, administrador, empleado, cliente)?

10 respuestas

9-¿Qué base de datos crees que sería la mejor opción para este proyecto?

10 respuestas

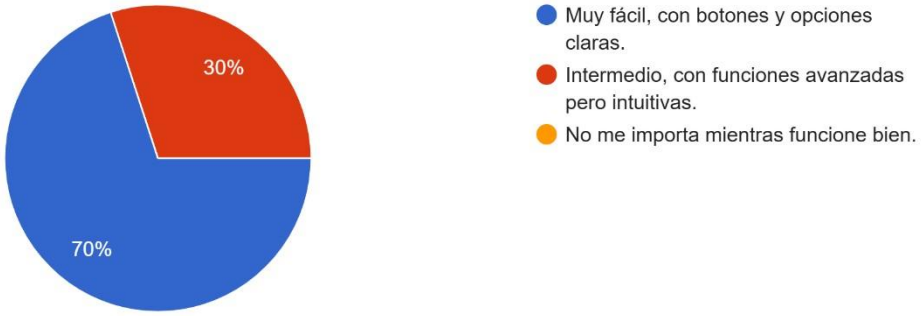
10-¿Qué nivel de seguridad consideras necesario para este sistema?

10 respuestas

-Usuarios

1. ¿Qué tan fácil de usar considera que debe ser un sistema de ventas para su tienda?

10 respuestas



2. ¿Qué tan útil le parecería un módulo para registrar compras y pagos a proveedores?

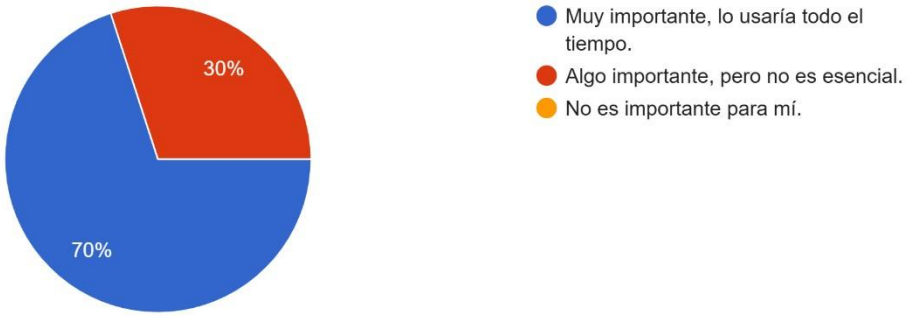
10 respuestas

3. ¿Preferiría que el sistema estuviera disponible en una página web?

10 respuestas

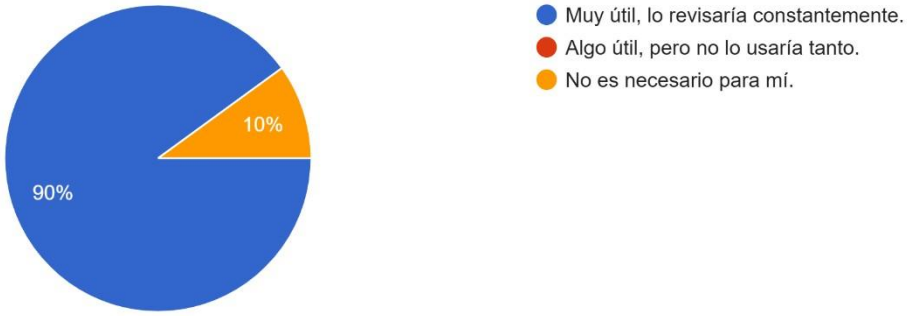
4. ¿Qué tan importante es para usted que el sistema tenga un apartado donde pueda visualizar y buscar sus productos fácilmente?

10 respuestas



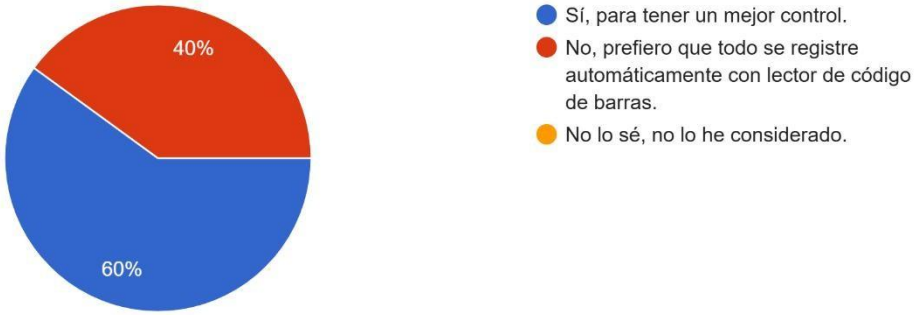
5. ¿Qué tan útil sería para usted un apartado que muestre las ganancias obtenidas por día?

10 respuestas



6. ¿Le gustaría poder registrar manualmente entradas y salidas de dinero en la tienda?

10 respuestas



7. ¿Qué tan importante sería para usted que el sistema registre las ventas realizadas por día?

10 respuestas

8. ¿Le parecería útil que el sistema le avise cuando un producto tenga pocas existencias?

10 respuestas

9.¿En qué dispositivo le gustaría usar este sistema?

10 respuestas

Reporte de conclusión

Para terminar, el desarrollo de este software representa un avance significativo en la gestión de inventario y ventas para las tiendas de abarrotes, en este caso específicamente a “Abarrotes Silvia”. A través de una interfaz intuitiva, funciones especializadas y un sistema de notificaciones para los productos que estén por terminarse, se logra un control más eficiente y preciso de los productos y transacciones. La implementación de esta herramienta no solo optimiza los procesos internos, sino que también permite una mejor toma de decisiones basada en datos actualizados. Aunque existen limitaciones, como la dependencia de internet y la falta de integración con sistemas contables externos, el software cumple con su propósito de mejorar la eficiencia operativa y reducir las pérdidas, y que no se compre producto de más o de manos a los proveedores. En el futuro, se planean considerar mejoras y expansiones para adaptar el sistema a nuevas necesidades y tecnologías emergentes.

11. Descripción de la solución propuesta (Cronograma de actividades, paso a paso explicando el front-end y el Back-end)

		Semana																
Actividades		1	2	3	4	5	6	7	8	9	10	11	12		13	14	15	Responsable
Requerimientos		27-31 Ene	3-7 Feb	10-14 Feb	17-21 Feb	24-28 Feb	3-7 Mar	10-14 Mar	17-21 Mar	24-28 Mar	31 Mar- 4 Abr	7-11 Abr	14-18 Abr	21-25 Abr	28 Abr- 2 May	5-9 May	12-16 May	
Entrevista	P																	Creado por Daniela Sepúlveda Gómez
	R	x																Creado por Daniela Sepúlveda Gómez
Contrato	P																	Creado por Daniela Sepúlveda Gómez
	R																	Creado por Daniela Sepúlveda Gómez
Diagrama	P																	Creado por Daniela Sepúlveda Gómez
	R	x																Creado por Daniela Sepúlveda Gómez
Cotización(alcance, tiempo, costo)	P																	Creado por Daniela Sepúlveda Gómez
	R		x	x														Creado por Daniela Sepúlveda Gómez
Planeación y desarrollo																		
Diseño de la base de datos	P																	Creado por Daniela Sepúlveda Gómez
	R		x	x														Creado por Daniela Sepúlveda Gómez
Desarrollo de Front-End	P																	Creado por Daniela Sepúlveda Gómez
	R			x	x	x	x	x	x	x								Creado por Daniela Sepúlveda Gómez
Desarrollo de Back-End	P																	Creado por Daniela Sepúlveda Gómez
	R			x	x	x	x	x	x	x								Creado por Daniela Sepúlveda Gómez
Pruebas																		
Caja blanca	P																	Creado por Daniela Sepúlveda Gómez
	R																	Creado por Daniela Sepúlveda Gómez
Caja negra	P																	Creado por Daniela Sepúlveda Gómez
	R																	Creado por Daniela Sepúlveda Gómez
Sobre carga	P																	Creado por Daniela Sepúlveda Gómez
	R																	Creado por Daniela Sepúlveda Gómez
Implementación																		
Entrega del sistema	P																	Creado por Daniela Sepúlveda Gómez
	R																	Creado por Daniela Sepúlveda Gómez
Documentación	P																	Creado por Daniela Sepúlveda Gómez
	R																	Creado por Daniela Sepúlveda Gómez

Sistema

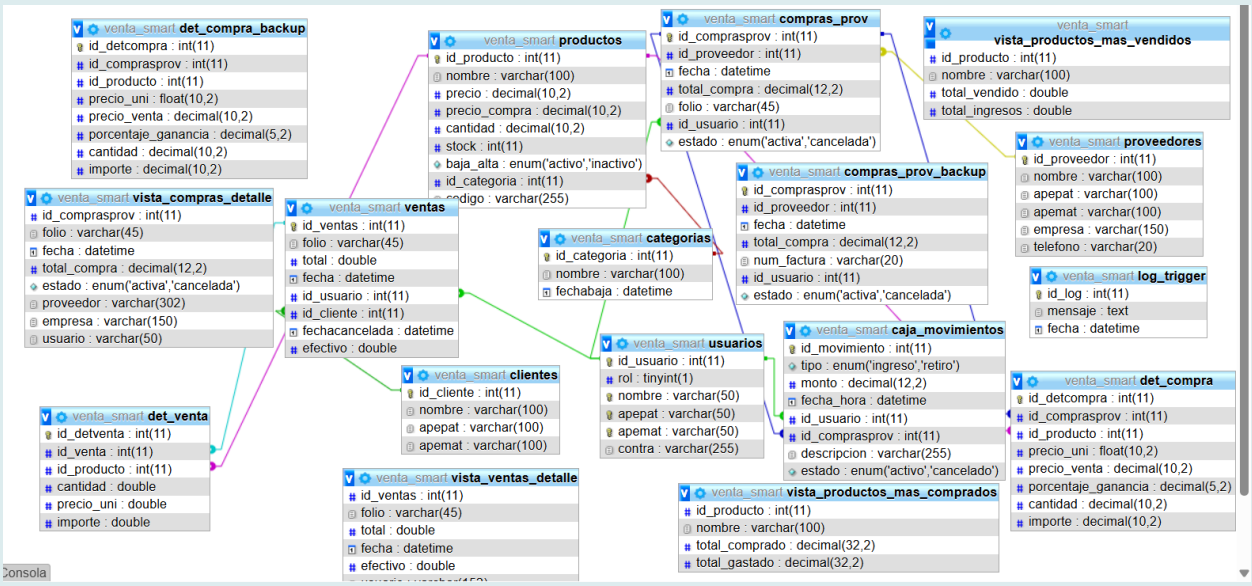
Base de datos, front-back, manual de usuario, manual de instalación, pruebas e instalación.

Base de datos

La base de datos está diseñada para las necesidades del negocio “Abarrotes Silvia” para que cumpliera con todos los requisitos pedidos por la gerente general.

Estas son las tablas:

```
mysql> use venta_smart;
Database changed
mysql> show tables;
+-------------------------------+
| Tables_in_venta_smart         |
+-------------------------------+
| caja_movimientos              |
| categorias                    |
| clientes                      |
| compras_prov                  |
| compras_prov_backup           |
| det_compra                    |
| det_compra_backup             |
| det_venta                     |
| log_trigger                   |
| productos                     |
| proveedores                   |
| usuarios                      |
| ventas                        |
| vista_compras_detalle         |
| vista_productos_mas_comprados |
| vista_productos_mas_vendidos  |
| vista_ventas_detalle          |
+-------------------------------+
17 rows in set (0.20 sec)
```



Estas son las descripciones de cada una:

```
mysql> describe caja_movimientos;
+----------------+---------------------------+------+-----+---------+----------------+
| Field          | Type                      | Null | Key | Default | Extra          |
+----------------+---------------------------+------+-----+---------+----------------+
| id_movimiento  | int(11)                   | NO   | PRI | NULL    | auto_increment |
| tipo           | enum('ingreso','retiro')  | NO   |     | NULL    |                |
| monto          | decimal(12,2)             | NO   |     | NULL    |                |
| fecha_hora     | datetime                  | NO   |     | NULL    |                |
| id_usuario     | int(11)                   | NO   | MUL | NULL    |                |
| id_comprasprov | int(11)                   | YES  | MUL | NULL    |                |
| descripcion    | varchar(255)              | YES  |     | NULL    |                |
| estado         | enum('activo','cancelado') | YES  |     | activo  |                |
+----------------+---------------------------+------+-----+---------+----------------+
8 rows in set (0.00 sec)
```

Registra todos los movimientos de dinero en caja, tanto ingresos como retiros.

Guarda monto, tipo de movimiento (ingreso o retiro), fecha, usuario que hizo el movimiento, y la relación con compras a proveedores si aplica.

```
mysql> describe categorias;
+--------------+--------------+------+-----+---------+----------------+
| Field        | Type         | Null | Key | Default | Extra          |
+--------------+--------------+------+-----+---------+----------------+
| id_categoria | int(11)      | NO   | PRI | NULL    | auto_increment |
| nombre       | varchar(100) | NO   |     | NULL    |                |
| fechabaja    | datetime     | YES  |     | NULL    |                |
+--------------+--------------+------+-----+---------+----------------+
3 rows in set (0.00 sec)
```

Contiene las categorías o clasificaciones de productos.

Sirve para organizar productos en grupos (por ejemplo, bebidas, alimentos, limpieza).

```
mysql> describe clientes;
+------------+--------------+------+-----+---------+----------------+
| Field      | Type         | Null | Key | Default | Extra          |
+------------+--------------+------+-----+---------+----------------+
| id_cliente | int(11)      | NO   | PRI | NULL    | auto_increment |
| nombre     | varchar(100) | NO   | MUL | NULL    |                |
| apepat     | varchar(100) | NO   |     | NULL    |                |
| apemat     | varchar(100) | NO   |     | NULL    |                |
+------------+--------------+------+-----+---------+----------------+
4 rows in set (0.00 sec)
```

Información de los clientes del sistema.

Datos básicos como nombre, apellido paterno y materno para identificar y gestionar a los clientes.

```
mysql> describe compras_prov;
+----------------+----------------------------+------+-----+---------+----------------+
| Field          | Type                       | Null | Key | Default | Extra          |
+----------------+----------------------------+------+-----+---------+----------------+
| id_comprasprov | int(11)                    | NO   | PRI | NULL    | auto_increment |
| id_proveedor   | int(11)                    | NO   | MUL | NULL    |                |
| fecha          | datetime                   | NO   | MUL | NULL    |                |
| total_compra   | decimal(12,2)              | NO   |     | NULL    |                |
| folio          | varchar(45)                | YES  |     | NULL    |                |
| id_usuario     | int(11)                    | NO   | MUL | NULL    |                |
| estado         | enum('activa','cancelada') | NO   |     | activa  |                |
+----------------+----------------------------+------+-----+---------+----------------+
7 rows in set (0.00 sec)
```

Registra las compras realizadas a los proveedores.

Contiene fecha, proveedor, total, folio, usuario que hizo la compra y estado de la compra (activa o cancelada).

```
mysql> describe compras_prov_backup;
+----------------+----------------------------+------+-----+---------+----------------+
| Field          | Type                       | Null | Key | Default | Extra          |
+----------------+----------------------------+------+-----+---------+----------------+
| id_comprasprov | int(11)                    | NO   | PRI | NULL    | auto_increment |
| id_proveedor   | int(11)                    | NO   | MUL | NULL    |                |
| fecha          | datetime                   | NO   | MUL | NULL    |                |
| total_compra   | decimal(12,2)              | NO   |     | NULL    |                |
| num_factura    | varchar(20)                | YES  |     | NULL    |                |
| id_usuario     | int(11)                    | NO   | MUL | NULL    |                |
| estado         | enum('activa','cancelada') | NO   |     | activa  |                |
+----------------+----------------------------+------+-----+---------+----------------+
```

7 rows in set (0.00 sec)

Copia de seguridad o respaldo de las compras a proveedores.

Similar a compras_prov, pero con un campo adicional num_factura, usado para guardar el número de factura oficial.

```
mysql> describe det_compra;
+---------------------+---------------+------+-----+---------+----------------+
| Field               | Type          | Null | Key | Default | Extra          |
+---------------------+---------------+------+-----+---------+----------------+
| id_detcompra        | int(11)       | NO   | PRI | NULL    | auto_increment |
| id_comprasprov      | int(11)       | NO   | MUL | NULL    |                |
| id_producto         | int(11)       | NO   | MUL | NULL    |                |
| precio_uni          | float(10,2)   | NO   |     | NULL    |                |
| precio_venta        | decimal(10,2) | NO   |     | NULL    |                |
| porcentaje_ganancia | decimal(5,2)  | NO   |     | NULL    |                |
| cantidad            | decimal(10,2) | NO   |     | NULL    |                |
| importe             | decimal(10,2) | NO   |     | NULL    |                |
+---------------------+---------------+------+-----+---------+----------------+
8 rows in set (0.00 sec)
```

Detalle de cada compra realizada a proveedores, con información por producto.

Registra productos comprados, cantidad, precios unitarios, precio de venta calculado, porcentaje de ganancia y el importe total por producto.

```
mysql> describe det_compra_backup;
+---------------------+---------------+------+-----+---------+----------------+
| Field               | Type          | Null | Key | Default | Extra          |
+---------------------+---------------+------+-----+---------+----------------+
```

```
| id_detcompra        | int(11)       | NO   | PRI | NULL    | auto_increment |
| id_comprasprov      | int(11)       | NO   | MUL | NULL    |                |
| id_producto         | int(11)       | NO   | MUL | NULL    |                |
| precio_uni          | float(10,2)   | NO   |     | NULL    |                |
| precio_venta        | decimal(10,2) | NO   |     | NULL    |                |
| porcentaje_ganancia | decimal(5,2)  | NO   |     | NULL    |                |
| cantidad            | decimal(10,2) | NO   |     | NULL    |                |
| importe             | decimal(10,2) | NO   |     | NULL    |                |
+---------------------+---------------+------+-----+---------+----------------+
8 rows in set (0.00 sec)
```

Copia de seguridad del detalle de compra, similar a det_compra.

Mantiene un respaldo de cada detalle de compra con los mismos campos.

```
mysql> describe det_venta;
+-------------+---------+------+-----+---------+----------------+
| Field       | Type    | Null | Key | Default | Extra          |
+-------------+---------+------+-----+---------+----------------+
| id_detventa | int(11) | NO   | PRI | NULL    | auto_increment |
| id_venta    | int(11) | NO   | MUL | NULL    |                |
| id_producto | int(11) | NO   | MUL | NULL    |                |
| cantidad    | double  | YES  |     | NULL    |                |
| precio_uni  | double  | YES  |     | NULL    |                |
| importe     | double  | YES  |     | NULL    |                |
+-------------+---------+------+-----+---------+----------------+
6 rows in set (0.00 sec)
```

Detalle de cada venta realizada, con productos vendidos y cantidades.

Guarda la relación de productos vendidos por cada venta, cantidades, precio unitario y importe.

```
mysql> describe log_trigger;
+---------+----------+------+-----+---------+----------------+
| Field   | Type     | Null | Key | Default | Extra          |
+---------+----------+------+-----+---------+----------------+
| id_log  | int(11)  | NO   | PRI | NULL    | auto_increment |
| mensaje | text     | YES  |     | NULL    |                |
| fecha   | datetime | YES  |     | NULL    |                |
+---------+----------+------+-----+---------+----------------+
3 rows in set (0.00 sec)
```

Tabla para almacenar logs o registros de eventos automáticos en la base de datos, posiblemente generados por triggers.

Guarda mensajes y fechas de los eventos para auditoría o depuración.

```
mysql> describe productos;
+---------------+---------------------------+------+-----+---------+----------------+
| Field         | Type                      | Null | Key | Default | Extra          |
+---------------+---------------------------+------+-----+---------+----------------+
| id_producto   | int(11)                   | NO   | PRI | NULL    | auto_increment |
| nombre        | varchar(100)              | NO   | MUL | NULL    |                |
| precio        | decimal(10,2)             | NO   |     | NULL    |                |
| precio_compra | decimal(10,2)             | YES  |     | NULL    |                |
| cantidad      | decimal(10,2)             | YES  |     | NULL    |                |
| stock         | int(11)                   | NO   |     | NULL    |                |
```

```
| baja_alta     | enum('activo','inactivo') | NO   | MUL | activo  |              |
| id_categoria  | int(11)                   | NO   | MUL | NULL    |              |
| codigo        | varchar(255)              | NO   |     | NULL    |              |
+---------------+---------------------------+------+-----+---------+----------------+
9 rows in set (0.00 sec)
```

Lista de productos disponibles en la tienda.

Incluye nombre, precio, precio de compra, cantidad en inventario, stock, estado activo o inactivo, categoría y código del producto.

```
mysql> describe proveedores;
+--------------+--------------+------+-----+---------+----------------+
| Field        | Type         | Null | Key | Default | Extra          |
+--------------+--------------+------+-----+---------+----------------+
| id_proveedor | int(11)      | NO   | PRI | NULL    | auto_increment |
| nombre       | varchar(100) | NO   | MUL | NULL    |                |
| apepat       | varchar(100) | NO   |     | NULL    |                |
| apemat       | varchar(100) | NO   |     | NULL    |                |
| empresa      | varchar(150) | NO   |     | NULL    |                |
| telefono     | varchar(20)  | NO   |     | NULL    |                |
+--------------+--------------+------+-----+---------+----------------+
6 rows in set (0.00 sec)
```

Información de los proveedores a los que se les realizan las compras de productos a la tienda.

Contiene datos como nombre, apellidos, empresa y teléfono para contacto.

```
mysql> describe usuarios;
+------------+--------------+------+-----+---------+----------------+
| Field      | Type         | Null | Key | Default | Extra          |
+------------+--------------+------+-----+---------+----------------+
| id_usuario | int(11)      | NO   | PRI | NULL    | auto_increment |
| rol        | tinyint(1)   | NO   |     | NULL    |                |
| nombre     | varchar(50)  | NO   | MUL | NULL    |                |
| apepat     | varchar(50)  | NO   |     | NULL    |                |
| apemat     | varchar(50)  | NO   |     | NULL    |                |
| contra     | varchar(255) | NO   |     | NULL    |                |
+------------+--------------+------+-----+---------+----------------+
6 rows in set (0.00 sec)
```

Usuarios que tienen acceso al sistema (empleados y administradores).

Incluye rol (tipo de usuario), nombre, apellidos y contraseña para autenticación.

```
mysql> describe ventas;
+----------------+-------------+------+-----+---------+----------------+
| Field          | Type        | Null | Key | Default | Extra          |
+----------------+-------------+------+-----+---------+----------------+
| id_ventas      | int(11)     | NO   | PRI | NULL    | auto_increment |
| folio          | varchar(45) | NO   |     | NULL    |                |
| total          | double      | YES  |     | NULL    |                |
| fecha          | datetime    | NO   |     | NULL    |                |
| id_usuario     | int(11)     | YES  | MUL | NULL    |                |
| id_cliente     | int(11)     | YES  | MUL | NULL    |                |
| fechacancelada | datetime    | YES  |     | NULL    |                |
| efectivo       | double      | NO   |     | NULL    |                |
```

```
+----------------+-------------+------+-----+---------+----------------+
```

8 rows in set (0.00 sec)

Registro general de las ventas realizadas.

Guarda folio, total de la venta, fecha, usuario que realizó la venta, cliente, estado (fecha de cancelación si aplica) y efectivo recibido.

```
mysql> describe vista_compras_detalle;
+----------------+---------------------------+------+-----+---------+-------+
| Field          | Type                      | Null | Key | Default | Extra |
+----------------+---------------------------+------+-----+---------+-------+
| id_comprasprov | int(11)                   | NO   |     | 0       |       |
| folio          | varchar(45)               | YES  |     | NULL    |       |
| fecha          | datetime                  | NO   |     | NULL    |       |
| total_compra   | decimal(12,2)             | NO   |     | NULL    |       |
| estado         | enum('activa','cancelada') | NO   |     | activa  |       |
| proveedor      | varchar(302)              | YES  |     | NULL    |       |
| empresa        | varchar(150)              | YES  |     | NULL    |       |
| usuario        | varchar(50)               | YES  |     | NULL    |       |
+----------------+---------------------------+------+-----+---------+-------+
```

8 rows in set (0.00 sec)

Vista que muestra un resumen de las compras a proveedores con detalles de proveedor y usuario.

Facilita consultas combinando varias tablas para ver compras con sus datos asociados.

```
mysql> describe vista_productos_mas_comprados;
+----------------+---------------+------+-----+---------+-------+
```

```
| Field          | Type          | Null | Key | Default | Extra |
+----------------+---------------+------+-----+---------+-------+
| id_producto    | int(11)       | NO   |     | NULL    |       |
| nombre         | varchar(100)  | NO   |     | NULL    |       |
| total_comprado | decimal(32,2) | YES  |     | NULL    |       |
| total_gastado  | decimal(32,2) | YES  |     | NULL    |       |
+----------------+---------------+------+-----+---------+-------+
4 rows in set (0.00 sec)
```

Vista que muestra los productos que más se han comprado, total comprado y total gastado.

Útil para análisis de inventario y compras.

```
mysql> describe vista_productos_mas_vendidos;
+----------------+--------------+------+-----+---------+-------+
| Field          | Type         | Null | Key | Default | Extra |
+----------------+--------------+------+-----+---------+-------+
| id_producto    | int(11)      | NO   |     | NULL    |       |
| nombre         | varchar(100) | NO   |     | NULL    |       |
| total_vendido  | double       | YES  |     | NULL    |       |
| total_ingresos | double       | YES  |     | NULL    |       |
+----------------+--------------+------+-----+---------+-------+
4 rows in set (0.00 sec)
```

Vista que muestra los productos más vendidos con total vendido e ingresos generados.

Ayuda a identificar los productos con mayor demanda.

```
mysql> describe vista_ventas_detalle;
+-----------+--------------+------+-----+---------+-------+
| Field     | Type         | Null | Key | Default | Extra |
+-----------+--------------+------+-----+---------+-------+
| id_ventas | int(11)      | NO   |     | 0       |       |
| folio     | varchar(45)  | NO   |     | NULL    |       |
| total     | double       | YES  |     | NULL    |       |
| fecha     | datetime     | NO   |     | NULL    |       |
| efectivo  | double       | NO   |     | NULL    |       |
| usuario   | varchar(152) | YES  |     | NULL    |       |
| cliente   | varchar(302) | YES  |     | NULL    |       |
+-----------+--------------+------+-----+---------+-------+
7 rows in set (0.00 sec)
```

Vista que muestra detalles de ventas, incluyendo datos del cliente, producto, cantidad y precio.

Utilizada para consultas rápidas y reportes de ventas detallados.

Dentro de ella cree dos procedimientos.

```
mysql> SELECT ROUTINE_NAME
    -> FROM INFORMATION_SCHEMA.ROUTINES
    -> WHERE ROUTINE_TYPE = 'PROCEDURE' AND ROUTINE_SCHEMA = DATABASE();
+-----------------+
| ROUTINE_NAME    |
+-----------------+
| insertarusuario |
| InsertarVenta   |
+-----------------+
```

Insertarusuario:

El procedimiento insertarusuario sirve para guardar un nuevo usuario en la tabla usuarios de la base de datos. Recibe cinco datos: el nombre, el apellido paterno, el apellido materno, el número de rol (que indica si es administrador, usuario, etc.) y la contraseña.

Al ejecutarlo, se inserta automáticamente un nuevo registro con esa información, sin tener que escribir todo el INSERT cada vez. Es una forma más rápida y ordenada de registrar usuarios. Sin embargo, es importante saber que la contraseña se guarda tal cual se escribe, lo que puede ser inseguro. Por eso, sería mejor aplicar algún tipo de encriptación antes de guardarla, para proteger mejor los datos del usuario.

InsertarVenta:

Este procedimiento se usa para registrar una venta de un producto en la base de datos. Primero, verifica si hay suficiente stock (cantidad disponible) del producto que se quiere vender. Si hay suficiente stock, entonces guarda la venta con la cantidad solicitada. Si no hay suficiente stock o el producto no existe, devuelve un mensaje de error diciendo que no se puede hacer la venta. Básicamente, ayuda a asegurarse de que no se vendan más productos de los que hay disponibles.

Dentro de ella también cree siete triggers:

```
mysql> SELECT COUNT(*) AS total_triggers
    -> FROM information_schema.TRIGGERS
    -> WHERE TRIGGER_SCHEMA = DATABASE();
+----------------+
| total_triggers |
+----------------+
|              7 |
+----------------+
1 row in set (0.01 sec)
```

```
mysql> SELECT TRIGGER_NAME
    -> FROM information_schema.TRIGGERS
    -> WHERE TRIGGER_SCHEMA = DATABASE();
+-----------------------------------+
| TRIGGER_NAME                      |
+-----------------------------------+
| actualizar_producto_despues_detalle |
| trg_devolver_productos            |
| restar_cantidad_producto          |
| actualizar_total_update           |
| actualizar_total_delete           |
| registrar_retiro_actualizado      |
| actualizar_total_compra           |
+-----------------------------------+
7 rows in set (0.01 sec)
```

Estas son las funciones de cada uno de ellos:

1. actualizar_producto_despues_detalle (Evento: INSERT en det_compra):

Cuando se agrega un detalle de compra (un producto comprado), este trigger actualiza el precio y la cantidad del producto en la tabla productos. También guarda un registro en log_trigger indicando si se actualizó el producto o si no se encontró. Sirve para mantener actualizado el inventario y precio del producto después de registrar una compra, y llevar un registro de estas acciones.

2. trg_devolver_productos (Evento: UPDATE en ventas):

Cuando se actualiza una venta y se marca como cancelada (se pone fecha de cancelación), este trigger devuelve la cantidad vendida de los productos a su stock original sumándola otra vez en la tabla productos. Sirve para asegurar que si una venta se cancela, el inventario se restaura correctamente.

3. restar_cantidad_producto (Evento: INSERT en det_venta):

Cada vez que se registra un detalle de venta (un producto vendido), este trigger reduce la cantidad disponible del producto en la tabla productos según la cantidad vendida. Sirve para disminuir el inventario automáticamente cuando se vende un producto.

4. actualizar_total_update (Evento: UPDATE en det_compra):

Cuando se actualiza un detalle de compra, este trigger recalcula el total de la compra principal en la tabla compras_prov sumando los subtotales de cada detalle. Sirve para mantener actualizado el total de la compra cuando cambian los detalles.

5. actualizar_total_delete (Evento: DELETE en det_compra):

Cuando se elimina un detalle de compra, este trigger recalcula y actualiza el total de la compra principal en compras_prov restando el detalle eliminado. Sirve para ajustar el total de la compra si se borra un producto de la compra.

6. registrar_retiro_actualizado (Evento: UPDATE en compras_prov):

Cuando se actualiza una compra y el total cambia de cero a un valor mayor, este trigger registra un movimiento de tipo “retiro” en la caja (caja_movimientos), indicando que se hizo un pago o retiro para esa compra.
Sirve para llevar control automático de los movimientos de caja cuando se registra un pago de compra.

7. actualizar_total_compra (Evento: INSERT en det_compra):

Cuando se inserta un nuevo detalle de compra, este trigger recalcula el total de la compra sumando los importes de todos los detalles relacionados y actualiza el total en compras_prov.
Sirve para mantener actualizado el total de la compra al agregar nuevos productos.

En total tengo cuatro vistas

```
mysql> SELECT TABLE_NAME AS vista_nombre
    -> FROM information_schema.VIEWS
    -> WHERE TABLE_SCHEMA = DATABASE();
+-------------------------------+
| vista_nombre                  |
+-------------------------------+
| vista_compras_detalle         |
| vista_productos_mas_comprados |
| vista_productos_mas_vendidos  |
| vista_ventas_detalle          |
+-------------------------------+
4 rows in set (0.05 sec)
```

Estas son las descripciones de cada una de ellas:

1. vista_compras_detalle:

Esta vista junta información de las compras que se han hecho a proveedores. Muestra detalles como el folio de la compra, la fecha, el total, el estado, el nombre completo del proveedor, la empresa del proveedor y el nombre del usuario que registró la compra. Sirve para facilita ver todo el detalle importante de las compras hechas a proveedores en un solo lugar, combinando datos de varias tablas.

2. vista_productos_mas_comprados:

Esta vista muestra cuáles productos se han comprado más en total. Suma la cantidad comprada y también el dinero total gastado en cada producto, agrupando por producto y ordenándolos para mostrar primero los más comprados. Ayuda a identificar qué productos son los más adquiridos por la empresa y cuánto se ha invertido en ellos.

3. vista_productos_mas_vendidos:

Esta vista muestra cuáles productos se han vendido más. Calcula la cantidad total vendida y los ingresos que se obtuvieron por cada producto, ordenándolos para mostrar primero los que más se venden.
Permite conocer cuáles son los productos más exitosos en ventas y cuánto dinero generan.

4. vista_ventas_detalle:

Esta vista combina la información de ventas con los datos del usuario que realizó la venta y el cliente que la compró. Muestra el folio, total de la venta, fecha, efectivo recibido, y nombres completos tanto del vendedor como del cliente. Facilita revisar el detalle completo de cada venta, incluyendo quién vendió y a quién se le vendió.

Y un log, porque no alcancé a hacer el otro:

El log_trigger sirve para registrar automáticamente información sobre cambios o problemas que ocurren cuando se modifican productos u otros datos a través de los triggers. Así puedes saber qué pasó y cuándo, sin tener que estar revisando manualmente cada cambio.

12. Análisis de Riesgos

Durante el desarrollo y futura operación del sistema para “Abarrotes Silvia”, se identificaron diversos riesgos que podrían afectar negativamente su funcionamiento, calidad o implementación.

Riesgo	Descripción	Impacto	Probabilidad	Estrategia de mitigación
Falla del hosting.	El servidor donde está alojado el sistema puede presentar caídas o interrupciones.	Alta.	Media.	Contratar un proveedor de hosting más confiable y realizar respaldos periódicos de la base de datos.
Pérdida de información.	Por errores humanos, fallos técnicos o falta de respaldo.	Alta.	Media.	Implementar copias de seguridad automáticas y periódicas.
Errores en el sistema.	Bugs o fallas de programación que afecten su funcionalidad.	Media.	Alta.	Realizar pruebas exhaustivas antes del poner el sistema en producción.
Dificultad de uso por parte del usuario.	El usuario no puede comprender el	Media.	Media.	Haber diseñado una interfaz intuitiva y ofrecer una

	uso de ciertas funciones.			breva capacitación.
Crecimiento del negocio no contemplado.	Nuevas necesidades (como varios usuarios o sucursales) no cubiertas por la versión actual.	Media.	Baja.	Diseñar el sistema de forma escalable para futuras actualizaciones.
Problemas con la conexión a internet.	El sistema depende de acceso a la red para funcionar en línea.	Media.	Media.	Tener una alternativa de respaldo local o asegurarse de contar con buena conexión.
Incompatibilidad con navegadores.	El sistema podría no funcionar adecuadamente en algunos navegadores.	Baja.	Media.	Probar y validar la interfaz en los navegadores más comunes (Chrome, Firefox, Edge).

13. Bibliografías

- Pressman, R. S. (2014). Ingeniería de Software: Un enfoque práctico. McGraw-Hill Education.
- Sommerville, I. (2011). Ingeniería del Software. Pearson Educación.
- W3Schools. (2024). https://www.w3schools.com Sitio de referencia para tecnologías web como HTML, CSS, JavaScript y PHP.
- PHP Manual. (2024). https://www.php.net/manual/es/ Documentación oficial del lenguaje de programación PHP.
- MySQL Documentation. (2024). https://dev.mysql.com/doc/ Documentación oficial para el gestor de base de datos MySQL.
- Bootstrap Official Docs. (2024). https://getbootstrap.com Documentación oficial para el framework de diseño responsivo Bootstrap.
- Chart.js Documentation. (2024). https://www.chartjs.org Librería utilizada para generar gráficos en el módulo de reportes.
- Font Awesome Docs. (2024). https://fontawesome.com/docs Biblioteca de íconos utilizada en la interfaz del sistema.
- SweetAlert2 Documentation. (2024). A beautiful, responsive, customizable, accessible (WAI-ARIA) replacement for JavaScript's popup boxes. Recuperado de https://sweetalert2.github.io/